

HOM Studio OS v2 - Robust Stack Strategy Addendum

Generated: 2026-05-25

This addendum updates the earlier PRD, blueprint, code-first UI strategy, and Codex implementation pack. It locks the most robust implementation strategy after evaluating Flask + React versus the previously recommended Next.js/TypeScript-first architecture.

Bottom Line

Use Next.js + React + TypeScript as the core product stack. Do not use Flask as the main backend. Add Python later only as an optional FastAPI worker/service for heavy document and AI processing if real performance needs justify it.

References Used for This Robust Stack Update

Re-check these before final production deployment because framework features, hosting limits, and pricing can change.

- Next.js App Router docs: <https://nextjs.org/docs/app>
- Next.js Server Actions and Mutations: <https://nextjs.org/docs/13/app/building-your-application/data-fetching/server-actions-and-mutations>
- FastAPI official docs: <https://fastapi.tiangolo.com/>
- FastAPI concurrency docs: <https://fastapi.tiangolo.com/async/>
- Flask async/await official docs: <https://flask.palletsprojects.com/en/stable/async-await/>
- Supabase database overview: <https://supabase.com/docs/guides/database/overview>
- Supabase pgvector docs: <https://supabase.com/docs/guides/database/extensions/pgvector>
- Supabase RLS docs: <https://supabase.com/docs/guides/database/postgres/row-level-security>
- Render regions and services docs: <https://render.com/docs/regions>
- Vercel CDN and functions docs: <https://vercel.com/docs/cdn>
- Motion.dev React docs: <https://motion.dev/docs/react>
- Storybook docs: <https://storybook.js.org/docs>
- OpenAI function calling and structured output docs: <https://developers.openai.com/api/docs/guides/function-calling>

20 - Robust Stack Decision

Purpose

This document locks the recommended stack for HOM Studio OS v2 after comparing the existing proposed build with Flask + React.

Final Verdict

Use this as the main strategy:

Core product:
Next.js + React + TypeScript
Supabase/Postgres
Render backend/API + Render worker
AI Gateway
Code-first design system
Storybook
Playwright

Do not rebuild the main system as:

Flask backend + React SPA frontend

Python is still useful, but only as a specialized worker/service for heavy document or AI processing later.

Why This Is the Most Robust Choice

HOM Studio OS v2 is not a simple CRUD app. It contains:

appointment operations
clinical case registry
session notes and note unlock approvals
financial ledger
practitioner commission
reimbursements
WhatsApp live chat
AI Business Agent
Knowledge Studio
behavior intelligence
file ingestion
background workers
approval workflow

A two-stack Flask + React architecture creates more boundaries for a solo developer: separate API contracts, CORS, auth handling, schema validation, OpenAPI discipline, deployment separation, and duplicate type definitions. That

is manageable for a team, but it adds avoidable complexity for a solo developer using Codex.

The robust strategy is to keep the product and dashboard in a single TypeScript-first ecosystem, while allowing a Python worker only where Python clearly wins.

Stack Lock

Frontend and BFF:

Next.js App Router

React

TypeScript strict mode

Tailwind

shadcn/ui

Radix UI

Motion.dev

Storybook

Database and platform data:

Supabase Postgres

Supabase Auth

Supabase Storage

Supabase pgvector

Supabase RLS

Backend/API:

Next.js route handlers and server actions for app-facing mutations

A dedicated Render service if API/workers need separation

Zod validation at every boundary

Background jobs:

Render worker

Postgres event_outbox first

pg-boss or Redis-backed queue when needed

AI:

AI Gateway package

structured outputs

function/tool calling

RAG over approved knowledge

multi-LLM routing later, not first

Optional Python:

FastAPI worker/service for document parsing, OCR, embeddings, and AI evals

Not Flask as the main app backend

No-Flask-As-Core Rule

Flask is not banned. It is simply not the best fit as the core backend for this product.

Use Flask only if:

- it is a tiny internal utility
- it has no high-concurrency external API calls
- it has no core transaction ownership
- it can be replaced without touching product logic

For production Python APIs, prefer FastAPI.

FastAPI Optional Service Rule

Add FastAPI only when one of these becomes real:

- Docling/Unstructured pipeline becomes heavy
- OCR/image parsing needs Python tools
- spreadsheet intelligence becomes complex
- embedding batch jobs slow down TypeScript workers
- RAG evaluation needs Python libraries
- ML/data science experiments need Python ecosystem

Do not add FastAPI in Phase 1 just because it sounds more powerful.

Source of Truth Rule

Supabase/Postgres remains the source of truth.

No worker, AI model, FastAPI service, or frontend component may become the source of truth for:

- appointments
- clinical notes
- finance
- payroll
- approvals
- client profile
- knowledge versions
- AI logs

Deployment Rule

Primary production path:

- Vercel: frontend/admin dashboard
- Render: backend/API and worker
- Render Key Value/Redis: cache/queue if needed

Supabase: database, auth, storage, vector
Optional VPS later: heavy AI/document worker only

Architecture Name

The official architecture name for the project is:

AI-native Modular Monolith
+ Selective Hexagonal Architecture
+ Code-first UI Design System
+ Event-driven Worker Layer
+ Optional Python AI/Document Worker

21 - Optional Python Service Strategy

Purpose

This document explains when and how Python should be introduced without making the project harder to maintain.

Bottom Line

Do not use Python as the main backend during Phase 1.

Use Python later as an isolated service for tasks where Python is clearly better than JavaScript/TypeScript.

Recommended Python Service Type

Use FastAPI, not Flask, for production-style Python services.

Reason:

FastAPI has stronger type-hint-first API design.
FastAPI creates OpenAPI docs automatically.
FastAPI is a better fit for async API calls and AI/document services.
Flask can still be used for tiny utilities, but not the core backend.

When to Add Python

Add Python only if these tasks become painful in TypeScript:

PDF parsing with Docling
OCR-heavy document processing
image understanding pre-processing
complex XLSX interpretation
embedding batch processing

RAG evaluation with Python-first libraries
custom ML/data science experiments
large offline analysis of chat behavior

Python Service Responsibilities

Allowed responsibilities:

```
parse_document(file_id)
extract_tables(file_id)
extract_images(file_id)
run_ocr(file_id)
generate_embeddings(source_id)
run_rag_eval(test_run_id)
summarize_large_batch(batch_id)
```

Forbidden responsibilities:

```
create appointment
reschedule appointment
edit clinical note
approve reimbursement
approve payroll
send WhatsApp blast without approval
modify financial ledger
change client package status
change permission roles
```

Communication Pattern

Use async job style:

```
Next.js/Backend creates job
↓
job stored in event_outbox or job_runs
↓
Python worker picks job or receives restricted API call
↓
Python writes result to safe result table or storage path
↓
Backend validates result
↓
Backend publishes result to product UI
```

Do not allow Python service to perform direct sensitive mutations.

Data Access Rule

Preferred:

Python receives signed file URL or scoped input payload.
Python returns structured result.
Main backend writes final state.

Avoid:

Python service has full database service-role access.
Python service can query all clients/finance/clinical notes freely.

If service-role is unavoidable, restrict by environment, network, and function-level allowlist.

API Contract Example

```
{
  "job_id": "job_123",
  "file_id": "file_456",
  "task": "parse_pdf",
  "source_scope": "internal_knowledge",
  "callback_url": "https://api.example.com/internal/jobs/job_123/result"
}
```

Result:

```
{
  "job_id": "job_123",
  "status": "completed",
  "parser": "docling",
  "text_blocks": 120,
  "tables": 4,
  "images": 8,
  "output_storage_path": "processed/job_123/result.json",
  "warnings": []
}
```

Folder Structure if Added

```
apps/
  web/
  worker/
  python-ai-worker/
    app/
      main.py
      routers/
      services/
      parsers/
      schemas/
      jobs/
```

```
tests/  
Dockerfile
```

Phase Decision

Phase 1: No Python service.

Phase 2: Add only if Knowledge Studio ingestion becomes slow.

Phase 3: Add dedicated Python worker for OCR/RAG evaluation if needed.

22 - Updated Robust Execution Order

Purpose

This is the updated build order after locking the robust stack decision.

Golden Rule

Do not start with Flask, microservices, or full multi-LLM automation.

Start with one stable TypeScript product foundation.

Phase 0 - Repository and Guardrails

Goal: make the repo safe for Codex.

Tasks:

1. Create Next.js app with TypeScript strict mode.
2. Add Tailwind, shadcn/ui, Radix UI, Motion.dev.
3. Add Storybook.
4. Add Biome or ESLint/Prettier.
5. Add Playwright.
6. Add folder structure for domain modules.
7. Add docs folder from this pack.
8. Add environment variable examples.
9. Add no-secret policy.
10. Add GitHub Actions basic checks.

Exit criteria:

```
npm run typecheck passes  
npm run lint passes  
npm run test placeholder passes  
Storybook opens locally  
Playwright smoke test runs
```

Phase 1 - Design System and App Shell

Goal: rebuild the HOM visual foundation in code.

Tasks:

1. Implement color tokens from HOM mood: off-white, charcoal, warm gold, muted green, soft red.
2. Create AppShell with dark sidebar and top search bar.
3. Create reusable MetricCard, DataTable, StatusBadge, EmptyState, ErrorState, LoadingSkeleton.
4. Create Storybook stories for all core UI components.
5. Build static Strategic Overview page with mock data.
6. Add responsive layout rules.
7. Add reduced-motion support.

Exit criteria:

No page-specific one-off styling for shared components.

Every component has a Storybook story.

Dashboard works with mock data.

Phase 2 - Core Data and RBAC

Goal: establish the operational source of truth.

Tasks:

1. Create Supabase schema migrations.
2. Add users, roles, permissions, user_roles.
3. Add clients, practitioners, services.
4. Add audit_logs.
5. Add RLS policies.
6. Add seed data for local/dev.
7. Add Zod schemas for inputs and outputs.

Exit criteria:

A non-authorized user cannot access finance/clinical data.

All sensitive actions create audit logs.

RLS smoke tests pass.

Phase 3 - Appointment Core

Goal: make operations reliable before dashboards get smarter.

Tasks:

1. Create appointments table and status history.
2. Build appointment creation form.
3. Add practitioner availability validation.
4. Add reschedule flow.
5. Add cancel/no-show/done statuses.

6. Add calendar/list views.
7. Add WhatsApp conversation link placeholder.

Exit criteria:

No double booking for same practitioner/time.
Every reschedule has history.
Every cancellation has reason.
Appointment UI has loading, empty, error, and success states.

Phase 4 - Clinical Core

Goal: protect clinical context and notes.

Tasks:

1. Create chronic case registry.
2. Create client condition flags.
3. Create session notes.
4. Create note lock/unlock workflow.
5. Add approval requests.
6. Add clinical audit logs.

Exit criteria:

Locked notes cannot be edited directly.
Unlock requests require reason and approval.
Clinical data is not sent raw to AI by default.

Phase 5 - Finance Core

Goal: fix financial consistency before AI CFO.

Tasks:

1. Create financial_ledger.
2. Create monthly_summary.
3. Create expenses.
4. Create therapist_commissions.
5. Create reimbursements.
6. Add report export placeholder.
7. Ensure dashboard cards and ledger use same period filter.

Exit criteria:

Financial summary and ledger reconcile.
No fake zero values when API fails.
Finance edits create audit logs.

Phase 6 - Worker and Automation Foundation

Goal: add async jobs without n8n as core.

Tasks:

1. Create event_outbox.
2. Create job_runs.
3. Create Render worker app.
4. Add retry/backoff.
5. Add dead-letter status.
6. Add internal job dashboard.

Exit criteria:

Failed job can be retried.

Worker failure does not break core transaction.

Job logs are visible to admin/developer.

Phase 7 - WhatsApp and Live Chat

Goal: build human-controlled AI communication.

Tasks:

1. Create conversations and messages.
2. Add inbound webhook handler.
3. Add Live Chat page.
4. Add manual intervention toggle.
5. Add AI draft placeholder.
6. Add safe reply policy.

Exit criteria:

Manual intervention disables auto-reply.

AI cannot reschedule without backend validation.

AI cannot diagnose or promise refunds.

Phase 8 - Knowledge Studio

Goal: let owner manage AI knowledge safely.

Tasks:

1. Add knowledge_sources.
2. Add file upload to Supabase Storage.
3. Add document processing job.
4. Add knowledge extraction review UI.
5. Add scope assignment.
6. Add publish and rollback.
7. Add test lab.

Exit criteria:

Uploaded knowledge is not active until published.
Every published knowledge version can be rolled back.
AI answer can show which source was used.

Phase 9 - AI Gateway

Goal: centralize all AI usage.

Tasks:

1. Create ai-gateway package.
2. Add provider interface.
3. Add one model first.
4. Add structured output schemas.
5. Add prompt versioning.
6. Add Langfuse or logging adapter.
7. Add cost and latency logs.

Exit criteria:

No direct model calls from UI components.
No direct model calls from random feature files.
Every AI call has prompt version, model, latency, cost estimate, and scope.

Phase 10 - Optional Python Service Decision

Goal: add Python only if justified.

Decision checklist:

Is document ingestion slow in TypeScript?
Do we need OCR/PDF parsing libraries better served by Python?
Do we need batch AI eval tools that are Python-first?
Can this be isolated from core transactions?

If yes, add FastAPI worker. If no, keep TypeScript worker.

23 - Codex Rules for Robust Stack

Purpose

This document gives Codex strict instructions so it does not accidentally create the wrong a

Hard Rules

```text

Do not introduce Flask as the main backend.  
Do not introduce FastAPI until explicitly requested in a later phase.  
Do not call OpenAI/Anthropic/Gemini directly from UI components.  
Do not let workers mutate sensitive domain data without backend validation.  
Do not build dashboards before source-of-truth tables exist.  
Do not create one-off UI components when a reusable component should exist.  
Do not add paid APIs unless behind environment variables.  
Do not put secrets in code or docs.  
Do not implement n8n as core transaction logic.

## Preferred Implementation Pattern

Use this pattern for every feature:

1. Define domain model.
2. Define database migration.
3. Define Zod schema.
4. Define service/use-case function.
5. Define route/server action.
6. Define UI component.
7. Add loading/empty/error/success states.
8. Add tests.
9. Add Storybook story if UI component.
10. Add audit log if sensitive.

## Folder Rules

|                           |                                       |
|---------------------------|---------------------------------------|
| apps/web/app              | = routes/pages/layouts                |
| apps/web/features         | = feature-specific UI and hooks       |
| apps/web/components/ui    | = generic reusable UI                 |
| apps/web/components/hom   | = HOM-specific reusable UI            |
| apps/web/components/agent | = AI/chat components                  |
| apps/worker               | = TypeScript background jobs          |
| packages/domain           | = business logic                      |
| packages/db               | = migrations, schema helpers, queries |
| packages/ai-gateway       | = all model calls                     |
| packages/shared           | = shared types and utilities          |

## AI Rules

All AI calls must go through packages/ai-gateway.  
Every AI call must log model, prompt version, feature, latency, token estimate, and result s  
AI must return structured output for classification/action proposals.  
AI may draft responses.  
AI may suggest actions.

AI may not directly execute sensitive actions.  
Sensitive actions require backend validation and often human approval.

## UI Rules

Every screen must have loading, empty, error, and success states.  
Every table must have filters/search if data can grow.  
Every dashboard metric must show period/source.  
Never show fake zero values for failed API calls.  
Use subtle Motion.dev animation only after layout is stable.  
Keep HOM visual identity: calm, premium, clinical, warm neutral.

## Python Rules

No Python in Phase 1.  
If Python is added later, use FastAPI or a queue worker.  
Python service must not own core data.  
Python must not bypass RBAC/RLS for sensitive actions.  
Python writes processing results, not final domain decisions.

## Deployment Rules

Frontend deploys to Vercel.  
Backend/API and worker deploy to Render.  
Supabase remains database/auth/storage/vector.  
Redis/Render Key Value is optional until queue/cache needs justify it.  
VPS is optional only for heavy document/AI worker later.

---

# 24 - Technology Stack Lockfile

This document is the plain-English stack lockfile. If there is a conflict between older docs and this document, this document wins.

## Main Stack

Frontend: Next.js + React + TypeScript  
UI: Tailwind + shadcn/ui + Radix UI  
Motion: Motion.dev  
Component docs: Storybook  
E2E tests: Playwright  
Database: Supabase Postgres  
Auth: Supabase Auth initially, with RBAC in app tables  
Storage: Supabase Storage  
Vector search: Supabase pgvector

Backend runtime: Next.js route handlers/server actions and/or Render API service  
Worker: Render TypeScript worker  
Queue: event\_outbox first; pg-boss or Redis later  
AI: centralized AI Gateway  
Deployment: Vercel + Render + Supabase

## Not Main Stack

Flask is not the main backend.  
FastAPI is not Phase 1.  
n8n is not core transaction logic.  
Microservices are not Phase 1.  
Kubernetes is not Phase 1.  
VPS is not Phase 1 except optional experimentation.

## Future Allowed Additions

FastAPI Python AI/document worker  
Dedicated VPS heavy worker  
Redis/Render Key Value  
Langfuse  
OpenTelemetry  
PostHog  
Sentry

## Why This Lockfile Exists

Codex and AI coding agents can easily over-expand a project. This file keeps implementation focused and prevents architecture drift.

Every time Codex proposes a new tool, ask:

Does it reduce complexity now?  
Does it replace an existing tool?  
Does it improve safety, performance, or maintainability?  
Can a beginner maintain it?  
Can it be removed later?

If the answer is unclear, do not add it.